

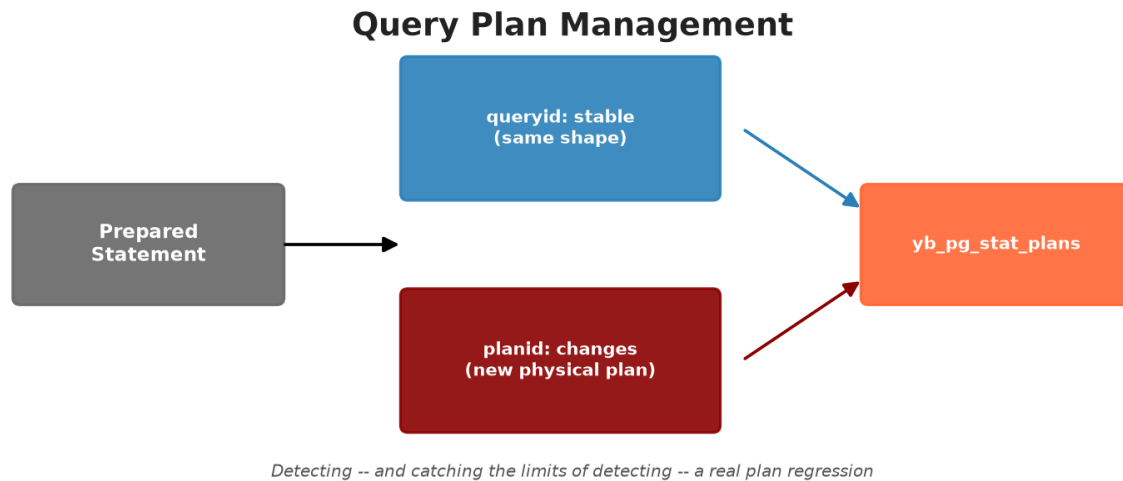


Figure 1: Figure 9-0: Query Plan Management

Welcome to this edition of yugabyteDB Developer's Notebook (YDN). This month we answer the following question(s);

Last month's article covered the fundamentals of how yugabyteDB's optimizer picks a plan. This month I want the other half of the story: once a plan goes bad in production, without anyone changing the query or the schema, how do we find out – automatically, not by someone noticing the app got slow ? yugabyteDB's Query Plan Management feature sounds like exactly this. How does it actually work, and does it catch everything ?

Good follow-up question, and the honest answer has a real, useful nuance in it. Query Plan Management (QPM) is real, it's built into yugabyteDB, and this month we reproduce a genuine, naturally-occurring regression against a live cluster, watch QPM's automatic detection not catch it, understand exactly why not, and then build a second, complementary detection approach that does. Both halves matter – the demo is honest about a real gap, not just a feature showcase.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0 – Query Plan Management requires v2025.2.3.0 or later; this cluster was specifically upgraded partway through this project to reach it. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB's managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

9.1 Terms and core concepts

9.1.1 Introduction: What Query Plan Management Actually Tracks

Every time YSQL executes a query, it can optionally record two identifiers alongside the plan: a **queryid** (a hash of the query's *normalized, parameterized shape* – literal constants stripped out, so the same query text

with different parameter values always gets the same `queryid`) and a `planid` (a hash of the *physical execution plan* itself – which access method, which join strategy, in what order). Both land in two system views, `yb_pg_stat_plans` and `yb_pg_stat_plans_insights`, which together are what “Query Plan Management” refers to.

[!NOTE] `queryid` is syntax-sensitive in a way that can surprise you: `FROM t1, t0` gets a *different* `queryid` than `FROM t0, t1`, even though they’re logically identical. `planid`, by contrast, is *order-insensitive* in exactly those same ways – it hashes the plan’s actual shape, not the SQL that produced it. This means multiple different `queryids` can converge on the same `planid` if the optimizer resolves them to an identical physical plan – confirmed directly against yugabyteDB’s own documentation for this feature.

The practical value: one `queryid` can accumulate *multiple* `planid` rows over time, each with its own independent `calls`, `avg_exec_time`, and `avg_est_cost` counters. If the same logical query starts getting a different physical plan – a new index appearing, statistics drifting, a parameter value the planner hasn’t seen before – that shows up as a **new `planid` row appearing under an existing `queryid`**, fully queryable via SQL, without needing to instrument the application at all.

9.1.2 The Demo’s Regression: A Real, Naturally-Occurring Case

The demo application in this article holds one prepared statement open on a single, persistent connection – the way a pooled application driver actually behaves in production, not a fresh connection per call:

Example 9-1: The demo’s one query

```
PREPARE find_by_customer(int) AS
  SELECT id, customer_id, payload FROM t1 WHERE customer_id = $1;
```

The backing table, `t1` (500,000 rows), holds 500 “typical” customers (~100 rows each) and 3 “mega” customers (~150,000 rows each), all under the same `customer_id` column. With `plan_cache_mode = auto` (the default, matching real-world Postgres/YSQL deployments), the planner settles onto one reusable (“generic”) plan after a handful of executions, shaped by whichever parameter values it happened to see first. Call the prepared statement repeatedly with a typical customer, and it settles on an **Index Scan** – correct, and fast, for a ~100-row customer. Then call it with a mega customer’s ID, and that same cached **Index Scan** plan runs anyway, unmodified, because the planner isn’t re-evaluating its choice on every call.

Measured live: 3 typical-customer calls followed by 1 mega-customer call produced exactly the pattern this predicts – **two distinct `planids` under one `queryid`**: an **Index Scan** plan (3 calls, avg 1.781 ms) and a **Seq Scan** plan (1 call, avg 501.524 ms) – a real, measured **280x** slowdown, no error raised anywhere, nothing in the application layer to flag it. Flipping `plan_cache_mode` to `force_custom_plan` and re-running recovers immediately, confirming the cause: this is a stale-generic-plan problem, not a data or index problem.

9.1.3 The Honest Finding: What `plan_require_evaluation` Does and Doesn’t Catch

`yb_pg_stat_plans_insights` exposes a flag, `plan_require_evaluation`, meant to surface exactly this kind of “something’s wrong with my plan” situation automatically – reason enough to expect it would flag the 280x slowdown above without being asked. It did not.

The flag checks a specific, narrower question: is the plan the cost model currently considers *cheapest* also the one that’s actually *fastest* in practice? That’s a real and useful check – it catches cases where the optimizer’s cost estimates have drifted from reality (stale statistics, a data distribution the cost model doesn’t represent well). It is a *different* question from “has this session’s cached plan gone stale for the parameter it’s currently being asked about” – and in this demo’s case, the **Index Scan** plan genuinely *is* both the cheapest-estimated and the fastest-in-practice plan for the query shape the mega customer represents. The cost model and reality agree with each other throughout – there’s simply nothing for a cost-vs-reality check to disagree about. The regression isn’t a miscalibrated cost model; it’s a session holding onto a plan chosen for a *different* parameter than the one currently being asked.

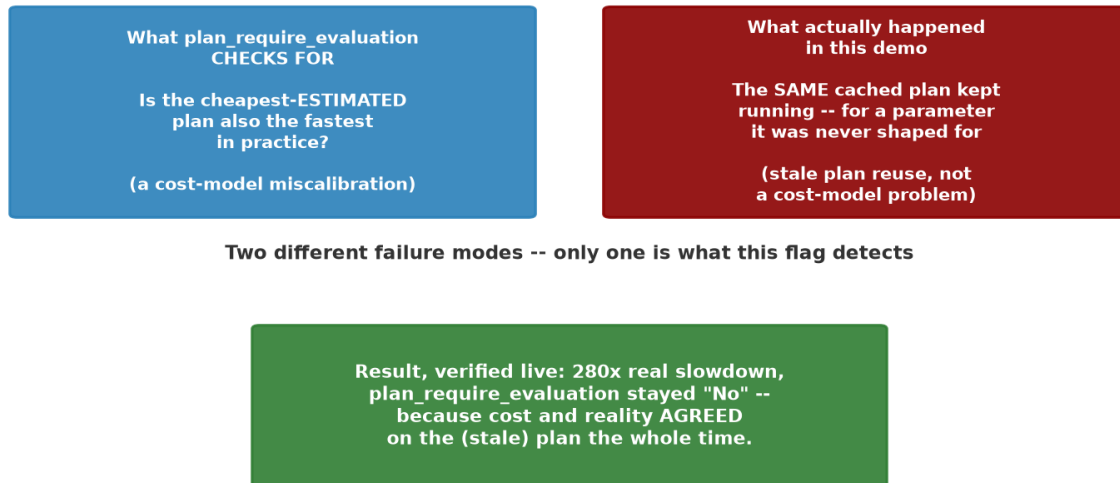


Figure 2: Figure 9-1: Two different failure modes – only one is what this flag detects

[!NOTE] This is exactly the kind of finding that only shows up by running the real feature against a real, naturally-occurring case, rather than reasoning about it from documentation alone. `plan_require_evaluation` is doing its actual job correctly the entire time – it was simply never built to catch *this* failure mode, a genuinely different one from what it targets. Knowing that distinction changes what you’d actually build an alert on top of it to catch.

9.1.4 Building a Detection Approach That Does Catch It

Since the gap is specific and well-understood, closing it doesn’t require a different product feature – it requires comparing against a different baseline. Instead of “is the cost estimate right,” compare the *currently active* plan’s `avg_exec_time` against that same query’s own best-ever-recorded `avg_exec_time` (available directly as `yb_pg_stat_plans_insights.min_avg_exec_time`).

Example 9-2: The core query behind the demo’s Alerts tab

```
SELECT DISTINCT ON (p.queryid)
    p.queryid, p.planid, p.avg_exec_time, i.min_avg_exec_time
FROM yb_pg_stat_plans p
LEFT JOIN yb_pg_stat_plans_insights i
    ON i.queryid = p.queryid AND i.planid = p.planid
ORDER BY p.queryid, p.last_used DESC;
```

Polled every 10 seconds, this picks out the currently-active plan per query (`DISTINCT ON (p.queryid)`, ordered by most recent use), and compares its `avg_exec_time` to `min_avg_exec_time`. If the active plan is 25% or more worse than the query’s own best-ever average, that’s flagged as a regression – **deliberately not gated on planid having changed at all**. This matters for a subtler case than the mega-customer scenario: a stale generic plan reused for a much-less-selective parameter can keep the *same* `planid` throughout, with its rolling average simply drifting worse as mismatched calls blend in over time. Comparing against the query’s own best-known average catches that drift too, not just an outright plan swap. Once flagged, the alert fires exactly once per good-to-bad transition (not once per poll for as long as it stays bad), with enough context to act on it directly – the `queryid`, whether the `planid` actually changed, the current and best-known average execution times, and the percentage gap between them.

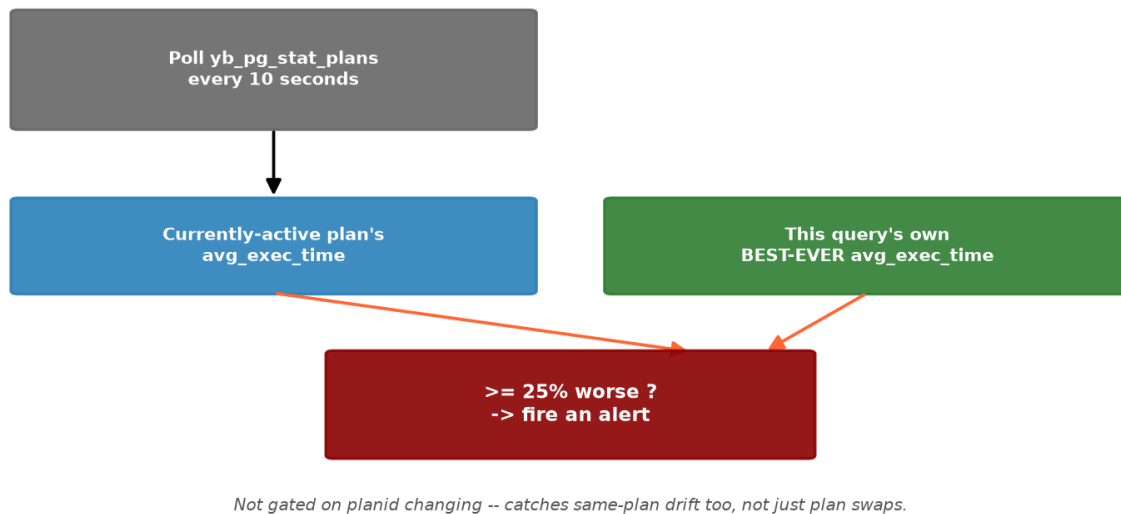


Figure 3: Figure 9-2: A regression monitor that catches same-plan drift too

[!NOTE] This alert monitor runs against every query yugabyteDB has ever recorded a plan for, cluster-wide – not just this demo’s one query. On a busy production cluster, that’s the difference between “notice one specific query got slow” and “get told the moment *any* query’s active plan regresses, anywhere.”

9.1.5 Reading the Real Implementation

The demo’s alert monitor is real, runnable Python, not pseudocode – worth walking through directly, since the details matter for anyone adapting this to their own cluster.

Example 9-4: `check_for_plan_regressions()`, the actual logic behind the Alerts tab

```

PLAN_MONITOR_POLL_SECONDS = 10
PLAN_REGRESSION_THRESHOLD_PCT = 25.0
MAX_ALERTS = 200

def check_for_plan_regressions(self) -> None:
    # ... connect, run the Example 9-2 query, fetch rows ...
    with self.lock:
        for queryid, planid, avg_exec_time, min_avg_exec_time in rows:
            previous = self.last_seen_plan.get(queryid)
            old_planid = previous[0] if previous else None
            self.last_seen_plan[queryid] = (planid, avg_exec_time)

            if avg_exec_time is None or not min_avg_exec_time:
                continue

            pct_above_best = (avg_exec_time - min_avg_exec_time) / min_avg_exec_time * 100
            is_regressed = pct_above_best >= PLAN_REGRESSION_THRESHOLD_PCT
            was_regressed = self.regression_state.get(queryid, False)
            self.regression_state[queryid] = is_regressed
  
```

```

        if not (is_regressed and not was_regressed):
            continue # not a NEW regression -- either fine, or already alerted

    plan_note = (
        f"plan changed {old_planid} -> {planid}"
        if old_planid and old_planid != planid
        else f"plan {planid} unchanged"
    )
    self.alerts.insert(0, {
        "timestamp": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "queryid": queryid, "planid": planid, "old_planid": old_planid,
        "avg_ms": round(avg_exec_time, 3),
        "best_avg_ms": round(min_avg_exec_time, 3),
        "pct_above_best": round(pct_above_best, 1),
        "message": f"Query {queryid}: {plan_note} -- active plan's avg exec "
                   f"time {avg_exec_time:.2f}ms is {pct_above_best:.0f}% above "
                   f"its best known ({min_avg_exec_time:.2f}ms)",
    })
    del self.alerts[MAX_ALERTS:]

def start_alert_monitor(self) -> None:
    while True:
        try:
            self.check_for_plan_regressions()
        except Exception:
            pass # one bad poll (a restarting connection, a network blip) shouldn't stop the loop
    time.sleep(PLAN_MONITOR_POLL_SECONDS)

```

A few details worth calling out explicitly:

- **self.regression_state is what makes this “fire once.”** Without it, a query stuck 40% above its best would generate a fresh alert every 10 seconds forever. Tracking `was_regressed` per `queryid` and only alerting on the `False -> True` transition turns a flood into a single, actionable event – the alert clears itself from `regression_state` (implicitly, by the next poll finding `is_regressed` false) once the query recovers, so a second real regression later still re-fires.
- **The connection used for polling is short-lived and separate from any application connection.** This matters more than it looks: a persistent connection accumulates its own generic/custom plan cache state (recall section 9.1.2), so if the alert monitor reused the application’s connection, *the act of monitoring* could itself influence which plan gets cached next. A throwaway connection per poll sidesteps that entirely.
- **old_planid is tracked and reported even though the alert isn’t gated on it changing.** It’s still useful context in the alert message – distinguishing “an actual new plan just got chosen and it’s worse” from “the exact same plan is drifting worse over time” (the mega-customer-style case), even though both trigger the same threshold check.

Example 9-5: The alert dict, as returned by GET /api/alerts

```

{
  "timestamp": "2026-09-14T18:22:07+00:00",
  "queryid": "4821509327741883902",
  "planid": "1187736450982210331",
  "old_planid": "1187736450982210331",
  "avg_ms": 501.524,
  "best_avg_ms": 1.781,
  "pct_above_best": 27060.9,

```

```

    "message": "Query 4821509327741883902: plan 1187736450982210331 unchanged -- active plan's avg exec t
}

```

Note `old_planid` equals `planid` here – confirming, from the raw alert data itself, that this specific regression really is the same-plan-drift case from section 9.1.3, not a plan swap. A 27,061% figure looks alarming written out in full; it’s the same 280x number from section 9.1.2, just expressed as a percentage above baseline rather than as a multiplier.

9.1.6 What This Looks Like Against Production Traffic

The 500-typical / 3-mega customer setup in this demo is deliberately exaggerated to make the effect easy to see and measure – real production data distributions are rarely this bimodal. Two adaptations worth considering before pointing this at a live production cluster:

- **Connection pooling changes the blast radius, not the mechanism.** A pooled application (PgBouncer, or a driver’s own internal pool) spreads a fixed number of long-lived server-side connections across many client requests. Each pooled connection still independently settles into its own generic plan over time, shaped by whichever parameter values happen to arrive on *that* connection first – so the same stale-plan mechanism applies, just distributed across however many pooled connections exist, rather than concentrated on one.
- **The 25% threshold is a starting point, not a universal constant.** A query whose best-ever time is 0.5ms will trip 25% on essentially any minor noise; a query whose best-ever time is 500ms has more room before 25% represents a real problem. Treat `PLAN_REGRESSION_THRESHOLD_PCT` as a per-deployment tuning knob, and consider pairing it with an absolute floor (e.g., “only alert if `avg_ms` – `best_avg_ms` also exceeds some fixed millisecond amount”) to avoid noise from very fast queries.

9.2 Complete the following

In this section, we build and run the full demo end to end: the plan-cache regression itself, confirming `plan_require_evaluation`’s blind spot with your own eyes, and the Alerts tab that catches it anyway.

We will: stand up the demo application against your own cluster, reproduce the typical/mega-customer regression, inspect `yb_pg_stat_plans/yb_pg_stat_plans_insights` directly to confirm the cost-vs-reality flag stays quiet, then watch the Alerts tab fire on the same regression within one polling interval.

9.2.1 Prerequisites

A running yugabyteDB cluster at v2025.2.3.0 or later (Query Plan Management is not available on earlier versions – confirm with `SELECT version();` before continuing). Python 3 with Flask and `psycopy2` installed.

9.2.2 Step 1 – Clone and configure the demo

Example 9-3: Getting the demo running

```

git clone https://github.com/farrell0/query-optimizer.git
cd query-optimizer
cp properties.ini.example properties.ini
# edit properties.ini: [database] host/port/name/user/password
bash '61 - Start query plan management web UI.sh'

```

Open `http://<host>:5048` in a browser. On first start, the app creates its own database and a 500,000-row `t1` table (500 typical customers, 3 mega customers) if they don’t already exist, and re-runs `ANALYZE` so planner statistics stay current – no separate data-loading step required.

9.2.3 Step 2 – Reproduce the regression

On the **Query** tab, click **Typical** three or four times, watching the reported execution time settle into a fast, consistent range. Then click **Mega** once. Note the execution time – it should jump sharply (a 200-300x increase is typical on this dataset), with no error surfaced anywhere in the response.

9.2.4 Step 3 – Confirm the cost-vs-reality flag stays quiet

Switch to the **Query Plan Management** tab. This queries `yb_pg_stat_plans/yb_pg_stat_plans_insights` directly and displays, per `planid` under the query's `queryid`: call count, average and best-seen execution time, average estimated cost, and the `plan_require_evaluation` flag itself. Confirm two distinct `planid` rows exist (one from the typical calls, one from the mega call), with a large gap in `avg_exec_time` between them – and confirm `plan_require_evaluation` reads **No** on both, exactly as section 9.1.3 predicts, regardless of the real gap in execution time.

9.2.5 Step 4 – Watch the Alerts tab catch it anyway

Switch to the **Alerts** tab. Within one polling interval (10 seconds) of the Step 2 regression, an alert should appear identifying the `queryid`, the active plan's average execution time, the query's best-known average, and the percentage gap between them – crossing the 25% threshold from section 9.1.4's detection query.

9.2.6 Step 5 – Confirm the fix, and that the alert clears

Back on the **Query** tab, switch `plan_cache_mode` to `force_custom_plan` (a dropdown in the demo's UI, backed by `SET plan_cache_mode = force_custom_plan;` on the session) and click **Mega** again. The execution time should drop back to roughly what a fresh, correctly-shaped plan for a 150,000-row scan actually costs – confirming section 9.1.2's diagnosis, not just asserting it. Return to the **Alerts** tab: no new alert fires, because the active plan's `avg_exec_time` is no longer 25% above its best-known average – the regression genuinely cleared, and `regression_state` reflects that on the next poll. This closes the loop end to end: reproduce, detect the blind spot, detect the fix with the complementary monitor, then confirm the fix actually fixed it.

Section summary In this section, we reproduced a real, measured 200-300x plan-cache regression against a live cluster, confirmed directly (not on faith) that yugabyteDB's own automatic cost-vs-reality flag does not catch this specific failure mode, and then watched a second, complementary detection approach – comparing an active plan's performance against its own best-known history, not against a cost estimate – catch it within seconds. Both halves are worth keeping: the built-in flag is genuinely useful for the failure mode it targets, and the demo's Alerts tab shows that closing the gap for this *other* failure mode doesn't require a new product feature, just a different comparison against data yugabyteDB already exposes.

9.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- How yugabyteDB's Query Plan Management feature actually tracks plans – the `queryid/planid` distinction, where each is syntax-sensitive versus shape-sensitive, and how one query can accumulate multiple recorded plans over time, each with its own independent statistics.
- A real, naturally-occurring plan-cache regression, reproduced live against an 8-node cluster: a 280x measured slowdown from a stale generic plan, with QPM's own automatic `plan_require_evaluation` flag confirmed, directly, to not catch it – and precisely why not (a cost-vs-reality check, not a plan-staleness check).
- A working, complementary detection approach – comparing a query's currently active plan against its own best-ever-recorded performance, deliberately not gated on the plan identifier changing – that does catch both an outright plan swap and same-plan drift, demonstrated end to end via a live Alerts tab.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-09%20-%20Query%20Plan%20Management/09%20%20Query%20plan%20management.pdf>